

Just Say No to More End-to-End Tests

Wednesday, April 22, 2015

by Mike Wacker

At some point in your life, you can probably recall a movie that you and your friends all wanted to see, and that you and your friends all regretted watching afterwards. Or maybe you remember that time your team thought they'd found the next "killer feature" for their product, only to see that feature bomb after it was released.

Good ideas often fail in practice, and in the world of testing, one pervasive good idea that often fails in practice is a testing strategy built around end-to-end tests.

Testers can invest their time in writing many types of automated tests, including unit tests, integration tests, and end-to-end tests, but this strategy invests mostly in end-to-end tests that verify the product or service as a whole. Typically, these tests simulate real user scenarios.

End-to-End Tests in Theory

While relying primarily on end-to-end tests is a bad idea, one could certainly convince a reasonable person that the idea makes sense in theory.

To start, number one on Google's list of [ten things we know to be true](#) is: "Focus on the user and all else will follow." Thus, end-to-end tests that focus on real user scenarios sound like a great idea. Additionally, this strategy broadly appeals to many constituencies:

- **Developers** like it because it offloads most, if not all, of the testing to others.
- **Managers and decision-makers** like it because tests that simulate real user scenarios can help them easily determine how a failing test would impact the user.
- **Testers** like it because they often worry about missing a bug or writing a test that does not verify real-world behavior; writing tests from the user's perspective often avoids both problems and gives the tester a greater sense of accomplishment.

End-to-End Tests in Practice

So if this testing strategy sounds so good in theory, then where does it go wrong in practice? To demonstrate, I present the following composite sketch based on a collection of real experiences familiar to both myself and other testers. In this sketch, a team is building a service for editing documents online (e.g., Google Docs).

Let's assume the team already has some fantastic test infrastructure in place. Every night:

1. The latest version of the service is built.
2. This version is then deployed to the team's testing environment.
3. All end-to-end tests then run against this testing environment.
4. An email report summarizing the test results is sent to the team.

The deadline is approaching fast as our team codes new features for their next release. To maintain a high bar for product quality, they also require that at least 90% of their end-to-end tests pass before features are considered complete. Currently, that deadline is one day away:

Days Left	Pass %	Notes
1	5%	Everything is broken! Signing in to the service is broken. Almost all tests sign in a user, so almost all tests failed.
0	4%	A partner team we rely on deployed a bad build to their testing environment yesterday.
-1	54%	A dev broke the save scenario yesterday (or the day before?). Half the tests save a document at some point in time. Devs spent most of the day determining if it's a frontend bug or a backend bug.
-2	54%	It's a frontend bug, devs spent half of today figuring out where.
-3	54%	A bad fix was checked in yesterday. The mistake was pretty easy to spot, though, and a correct fix was checked in today.
-4	1%	Hardware failures occurred in the lab for our testing environment.

-5	84 %	Many small bugs hiding behind the big bugs (e.g., sign-in broken, save broken). Still working on the small bugs.
-6	87 %	We should be above 90%, but are not for some reason.
-7	89.54 %	(Rounds up to 90%, close enough.) No fixes were checked in yesterday, so the tests must have been flaky yesterday.

Analysis

Despite numerous problems, the tests ultimately did catch real bugs.

What Went Well

- Customer-impacting bugs were identified and fixed before they reached the customer.

What Went Wrong

- The team completed their coding milestone a week late (and worked a lot of overtime).
- Finding the root cause for a failing end-to-end test is painful and can take a long time.
- Partner failures and lab failures ruined the test results on multiple days.
- Many smaller bugs were hidden behind bigger bugs.
- End-to-end tests were flaky at times.
- Developers had to wait until the following day to know if a fix worked or not.

So now that we know what went wrong with the end-to-end strategy, we need to change our approach to testing to avoid many of these problems. But what is the right approach?

The True Value of Tests

Typically, a tester's job ends once they have a failing test. A bug is filed, and then it's the developer's job to fix the bug. To identify where the end-to-end strategy breaks down, however, we need to think outside this box and approach the

problem from first principles. If we "focus on the user (and all else will follow)," we have to ask ourselves how a failing test benefits the user. Here is the answer:

A failing test does not directly benefit the user.

While this statement seems shocking at first, it is true. If a product works, it works, whether a test says it works or not. If a product is broken, it is broken, whether a test says it is broken or not. So, if failing tests do not benefit the user, then what does benefit the user?

A bug fix directly benefits the user.

The user will only be happy when that unintended behavior - the bug - goes away. Obviously, to fix a bug, you must know the bug exists. To know the bug exists, ideally you have a test that catches the bug (because the user will find the bug if the test does not). But in that entire process, from failing test to bug fix, value is only added at the very last step.

Stage	Failing Test	Bug Opened	Bug Fixed
Value Added	No	No	Yes

Thus, to evaluate any testing strategy, you cannot just evaluate how it finds bugs. You also must evaluate how it enables developers to fix (and even prevent) bugs.

Building the Right Feedback Loop

Tests create a feedback loop that informs the developer whether the product is working or not. The ideal feedback loop has several properties:

- **It's fast.** No developer wants to wait hours or days to find out if their change works. Sometimes the change does not work - nobody is perfect - and the feedback loop needs to run multiple times. A faster feedback loop leads to faster fixes. If the loop is fast enough, developers may even run tests before checking in a change.
- **It's reliable.** No developer wants to spend hours debugging a test, only to find out it was a flaky test. Flaky tests reduce the developer's trust in the test, and as a result flaky tests are often ignored, even when they find real product issues.

- **It isolates failures.** To fix a bug, developers need to find the specific lines of code causing the bug. When a product contains millions of lines of codes, and the bug could be anywhere, it's like trying to find a needle in a haystack.

Think Smaller, Not Larger

So how do we create that ideal feedback loop? By thinking smaller, not larger.

Unit Tests

Unit tests take a small piece of the product and test that piece in isolation. They tend to create that ideal feedback loop:

- **Unit tests are fast.** We only need to build a small unit to test it, and the tests also tend to be rather small. In fact, one tenth of a second is considered slow for unit tests.
- **Unit tests are reliable.** Simple systems and small units in general tend to suffer much less from flakiness. Furthermore, best practices for unit testing - in particular practices related to hermetic tests - will remove flakiness entirely.
- **Unit tests isolate failures.** Even if a product contains millions of lines of code, if a unit test fails, you only need to search that small unit under test to find the bug.

Writing effective unit tests requires skills in areas such as dependency management, mocking, and hermetic testing. I won't cover these skills here, but as a start, the typical example offered to new Googlers (or Nooglers) is how Google [builds](#) and [tests](#) a stopwatch.

Unit Tests vs. End-to-End Tests

With end-to-end tests, you have to wait: first for the entire product to be built, then for it to be deployed, and finally for all end-to-end tests to run. When the tests do run, flaky tests tend to be a fact of life. And even if a test finds a bug, that bug could be anywhere in the product.

Although end-to-end tests do a better job of simulating real user scenarios, this advantage quickly becomes outweighed by all the disadvantages of the end-to-end feedback loop:

	Unit	End-toEnd
Fast		
Reliable		
Isolates Failures		
Simulates a Real User		

Integration Tests

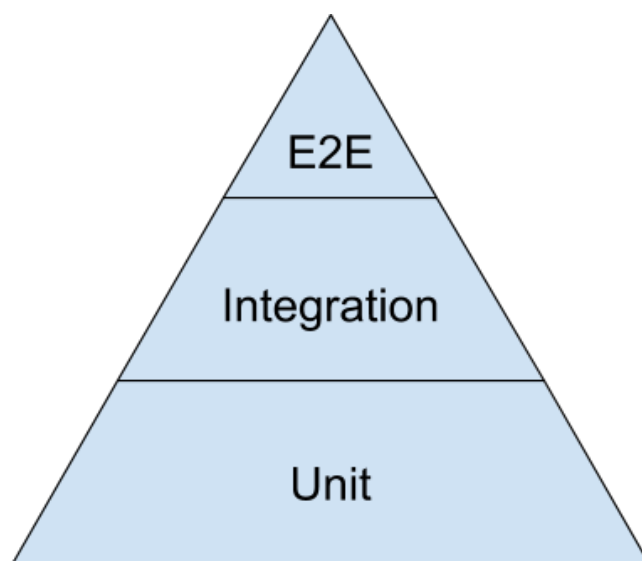
Unit tests do have one major disadvantage: even if the units work well in isolation, you do not know if they work well together. But even then, you do not necessarily need end-to-end tests. For that, you can use an integration test. An integration test takes a small group of units, often two units, and tests their behavior as a whole, verifying that they coherently work together.

If two units do not integrate properly, why write an end-to-end test when you can write a much smaller, more focused integration test that will detect the same bug?

While you do need to think larger, you only need to think a little larger to verify that units work together.

Testing Pyramid

Even with both unit tests and integration tests, you probably still will want a small number of end-to-end tests to verify the system as a whole. To find the right balance between all three test types, the best visual aid to use is the testing pyramid. Here is a simplified version of the [testing pyramid](#) from the opening keynote of the [2014 Google Test Automation Conference](#):



The bulk of your tests are unit tests at the bottom of the pyramid. As you move up the pyramid, your tests gets larger, but at the same time the number of tests (the width of your pyramid) gets smaller.

As a good first guess, Google often suggests a 70/20/10 split: 70% unit tests, 20% integration tests, and 10% end-to-end tests. The exact mix will be different for each team, but in general, it should retain that pyramid shape. Try to avoid these anti-patterns:

- **Inverted pyramid/ice cream cone.** The team relies primarily on end-to-end tests, using few integration tests and even fewer unit tests.
- **Hourglass.** The team starts with a lot of unit tests, then uses end-to-end tests where integration tests should be used. The hourglass has many unit tests at the bottom and many end-to-end tests at the top, but few integration tests in the middle.

Just like a regular pyramid tends to be the most stable structure in real life, the testing pyramid also tends to be the most stable testing strategy.



Labels: [Mike Wacker](#)

84 comments :



Unknown [April 23, 2015 at 2:45:00 AM PDT](#)

Meanwhile I share the opinion, I have problem with measuring the shape - just for curiosity, how you suggest to measure the size of unit/integration/E2E tests?

Comparing the coverage they have, a few E2E test can generate much higher coverage than several unit tests. Comparing numbers, and having n thousands of unit tests and having only <100 E2E tests, this would still be presented as pyramid (well in the given percentages), but the E2E part still may cause so many problems (time, effort, test env problems and value of the test), that we can say: we have the pyramid - but the goal is not achieved.

[Reply](#)

▼ [Replies](#)

Anonymous [April 23, 2015 at 11:22:00 AM PDT](#)

It can be hard to directly measure the unit/integration/E2E ratio for several reasons. However, deviating from the test pyramid has byproducts you can measure, such as increased test runtime and more flakes.

Let me use sorting algorithms and running time as an analogy. Quicksort can take $O(n^2)$ time in the worst case, but that worst case is rare enough that the expected runtime of quicksort is still $O(n \log n)$. However, if you use a sorting algorithm that always hit that $O(n^2)$ worst case, for example selection sort, then the expected runtime inflates from $O(n \log n)$ to $O(n^2)$.

Think of E2E tests as your worst case. If you have a small number E2E tests, the overall runtime of all your tests will still be quite reasonable. However, if you mostly use E2E tests, then your test runtime (and the number of test flakes) will inflate significantly.

[Reply](#)



Unknown [April 23, 2015 at 4:51:00 AM PDT](#)

I agree with the main idea, but it's nothing new. Let's look at V-model in testing.

I would add one thing: Before unit test it would be nice to perform a code deskcheck - statis testing - the first step in testing chain.

[Reply](#)

▼ [Replies](#)

Anonymous [April 25, 2015 at 3:24:00 PM PDT](#)

In my testing chain TDDing would be my first rather than a code deskcheck. If you want to test your code, why not simply do it beforehand? Might be quicker than pen&paper, more reliable, easier to reproduce, easier to extend .. and I sure find it more rewarding from a motivational perspective going from red to green than to write tests afterwards hoping that they turn green right away (and hoping that this 'green' is somewhat meaningful).

[Reply](#)



Random Kernel [April 23, 2015 at 5:18:00 AM PDT](#)

Hey Mike,

Thanks for the article. I think that sentence is good to be highlighted: "The exact mix will be different for each team, but in general, it should retain that pyramid shape."

A typical path for a test automation engineer is the following: 1) we do everything as close as possible to the real user's experience; 2) oh, well,

those tests are too slow and unstable; 3) let's move to unit tests; 4) oh, well, unit tests are good and green, but we do miss some important bugs here; 4) both unit and end-to-end tests are important. I don't mention integration tests here, since it's a too general term, and they may differ in size and value even within one project, not to say about different projects and teams.

Also, sometimes end-to-end tests are built upon API tests that may be considered as unit tests in some extent. So when we talk about percentage, we should take it into account, as well.

With all that in mind, here is my point: yes, the pyramid makes sense, but don't pay too much attention to 70/20/10 or anything like that. Think in term of `_your_` product, its specific, its challenges, and build your strategy and tactics on that.

[Reply](#)

▼ [Replies](#)

Anonymous [April 23, 2015 at 11:39:00 AM PDT](#)

I tend to take the opposite approach, starting with unit tests and only using larger tests when unit tests clearly are not sufficient.

As a useful thought experiment, pretend that you could only write 10 E2E tests, and ask yourself where those tests would go. As you said, each product has its own unique specifics and challenges, so the answer will be different for each product.

The testing pyramid can generalize to any product, and the problems associated with too many E2E tests will affect all products, but what will be unique for each product is where unit tests become insufficient and larger tests are needed.



Random Kernel [April 24, 2015 at 4:08:00 AM PDT](#)

Mike, I got your point. And this thought experiment seems to be useful. Let me share some thoughts, though.

Suppose your product has fundamental web interface. In this product,

1) part of UI operations can be executed without UI via API or command line interface. So you can write some basic unit tests for single operations, but are you going to verify UI operations, as well, to make sure, say, that not only core operations are successful, but also the changes made in the browser are delivered to the core functions? Also, some operations in UI may require preliminary steps. Each step may be considered as a test itself, but the real value comes from the whole chain of steps, because each step can be successful, and the chain is not. What if you guesses about necessary E2E tests for you products are wrong, and with all your formal unit tests coverage you miss important scenarios?

2) part of the other operations are intended for run in UI by their nature. Say, your product opens RDP session to some computer in your browser and run some UI-based operations there. Will you be satisfied with some mocks/stubs imitating remote computer behavior, or will you try to handle real sessions, as well?

You say that E2E tests are not fast, not reliable and hard to debug. But what if you are able to make them sufficiently fast, reliable, easy to implement and change when necessary, and you can easily understand test failures by their results? Will you say yes in that case?

I still agree with the concept of the testing pyramid, though.



br11 February 10, 2017 at 6:41:00 AM PST

I agree with Mike's idea and I'd like to contribute with one more argument.

Requirements can be categorized as Concepts, Facts or Rotines(processes). "Customer" is a concept. concepts might or might not be described by stating facts among other things. "A purchase is made by a customer" is a fact. Facts links two or more concepts. "After a purchase is made, the customer data are updated" is a process. A modification in a concept is usually demanded by a disrupting change in the business scenario. A modification in a Fact is usually demanded by a structural change in the business scenario. A modification in a process can be demanded by many sorts of things including the weather forecast.

It's possible to make minimal and gradual changes to an application flow (the process) with no negative impact in the user experience but even these small changes are likely to break the E2E automated tests. Maybe we need now to start thinking about some adaptive feature for the E2E implementation if not available yet.

to handle that in a real scenario I implemented an additional execution mode - Human Assisted - that before failing, asks the human assistant to make a change to the test case in order to it to stay succeeding. By doing this we could reach 40% E2E automated test coverage for a mobile banking application in a way that our client accepted. It represented 200 of a total of 500 application flows covered by E2E automated test.

[Reply](#)

Anonymous [April 23, 2015 at 11:24:00 AM PDT](#)

Yes, we still do. If you're trying to sell the testing pyramid to someone, using small/medium/large instead of unit/integration/E2E may make it an easier sell.

[Reply](#)



Unknown [April 24, 2015 at 11:34:00 PM PDT](#)

You should mention [FIRST](#) properties of unit tests.

FIRST should be applied to all tests much as possible, but bigger the scope, the harder it gets.

[Reply](#)



Ran Tavory [April 25, 2015 at 10:47:00 PM PDT](#)

Tests, as well as monitoring of all sorts (app level monitoring, host, user, kpi) are all part of the immune system of your software IMO .

I agree with the 70/20/10 approach but on top of the pyramid I would add another pyramid of monitoring. I argue that well thought out monitoring is more effective than tests in many cases, particularly in CD (continuous deployment) where MTTR (mean time to recovery) is far more important

than MTBF (mean time bw failures)

I'd go with 50/50 bw testing and monitoring time investment wise, at least in CD scenario.

BTW having to wait for tests (any test) to run at night doesn't make sense in many cases anyway, CD included.

[Reply](#)



Ran Tavory April 26, 2015 at 12:24:00 AM PDT

Coming from the CD (Continuous Deployment) perspective, I think things are a little different.

With CD the complete "immune system" means that monitoring (different types of monitors) are part of the immune system aside tests and they complement the tests (other components in the immune system are code review, static code analysis etc).

Interestingly, monitoring resembles testing in many ways, so you'd have application level monitoring, which usually are similar in scope to unit tests - they usually monitor individual in-process complements (e.g. size of internal memory buffer, operations/sec etc), you have host level monitoring (CPU, disk etc), which is similar in concept to integration tests and you have KPI monitoring (e.g. # daily active users etc) which takes the user perspective and is similar to E2E tests.

The picture would not be whole if you don't mention monitoring since, IMO monitoring come on the expense of testing - developers either invest time in tests or in monitoring (or split their efforts b/w these two)

I would argue that, at least in CD where MTTR (Mean Time to Recovery) is far more important than MTBF (Mean Time Between Failures), monitoring take precedence over tests. I would draw yet another pyramid - a monitoring pyramid - on top of the testing pyramid such that 70% is application level monitoring, 20% host monitoring and 10% KPI. And the entire effort b/w tests and monitoring should be split 50/50 (or some other number that makes sense for your use case - in some cases it's 90/10).

Again, I'm speaking from the perspective of CD - which may or may not apply to some google systems, but many dev organizations tend to like it.

BTW speaking about putting the user in the center, delivering value fast and being able to verify the value with actual users in matter of hours - the core value of CD - fast feedback (including the user in the feedback loop) - *is* putting the user in the center.

BTW2, a feedback loop needs to be in the order of a few hours at most (minutes sometimes), *including actual users* in the loop, not just automated tests. As such - running E2E tests during the night simply makes no sense.

[Reply](#)

▼ [Replies](#)

Anonymous [April 28, 2015 at 11:35:00 AM PDT](#)

Monitoring was not in scope for this blog post, but I do agree that monitoring is important, and that good monitoring will catch bugs that even good tests miss. There is a trade-off at times between monitoring and testing as you said, but they're not always mutually exclusive.

Monitoring is not particular useful, for example, if the code for your service doesn't even build. And if all your tests fail, you probably don't need monitoring to know that if you try to deploy that service in its current state, everything will break.

Your service doesn't need to be perfect before you deploy it, but it does need to meet some minimal quality bar before monitoring becomes useful. And tests are how you get it to that bar.

[Reply](#)



오장일 [April 27, 2015 at 1:32:00 AM PDT](#)

Hello

I would like to translate the contents of this blog in Korean on My Blog.
is it possible?

Have a good day.

[Reply](#)



Recurrence [April 27, 2015 at 1:50:00 PM PDT](#)

Sounds like Test Instability and Timeliness are your biggest beefs
(Addresses basically everything in 'What Went Wrong')

Just throw a thousand instances at the problem and have your results in (overhead + longest_test time). I've done something similar but with only 300 instances some years ago and we had E2E results in 12 minutes after EVERY commit.

Benefits:

- + You can isolate the test (General cause of instability)
- + Results are quick and can be traced to a specific commit
- + Comparatively little waiting period for results

That said, if your labs can't keep themselves up, you have no business in the E2E testing space.

[Reply](#)

▼ [Replies](#)

Anonymous [April 28, 2015 at 11:27:00 AM PDT](#)

Not everybody has the resources or funding to just throw a thousand instances at the problem, especially as they get more and more E2E tests. And building and deploying your service is typically part of that process of running E2E tests. For you, that doesn't seem to take a long time, but I've worked on teams that couldn't even build their service in 12 minutes, much less build, deploy, and run tests in 12 minutes. In short, I have doubts on whether that approach can scale beyond your specific situation.

But even if you can get it down to 12 minutes (and your E2E tests are not flaky), that's still slow compared to $< 1/10$ of a second for each unit test. If you want developers regular running some tests before they check-in, unit tests are the way to go.



Random Kernel [April 29, 2015 at 3:54:00 AM PDT](#)

Sure enough, developers would prefer running unit tests rather than E2E ones. But should this criteria be the most important? Please, consider two options:

1) running E2E tests takes more time than unit tests, but you have an opportunity to run these checks because you consider them

necessary;

2) you don't run E2E tests at all, or run only small amount of them (with percentage in testing pyramid you consider acceptable)

In both options, developers will only run unit tests, but in the first option, you will be able to have deeper coverage and more certainty in product quality. Well, in the worst case, developers will be informed on results after check-ins, but later better than never. In the best case (E2E tests are not long enough, and developers don't hurry with check-ins), you will kill two birds with one stone (better coverage and running tests before check-in).



Unknown April 30, 2015 at 12:56:00 AM PDT

To ignore the benefits of both end to end and unit testing is a mistake. That said, this article ignores some of the more difficult problems with Unit Tests. For one, they can create a barrier to refactoring especially if that refactoring breaks many tests and it's now the tests that are wrong, not the code that has been refactored. Moreover, if you need a significant amount of state in order to complete the test, a unit test is unlikely to give you the results.

With end to end tests it's likely they will not break when refactoring. If there is a failure in the end to end test than of course you need to isolate and that is when (smaller more focused) unit tests are immensely useful.

Anonymous May 1, 2015 at 12:44:00 PM PDT

It's not just a question of coverage and quality - it's a tradeoff between quality and velocity.

I sometimes will run my unit tests 12 times in the course of minutes, not once every 12 minutes. If my team takes over an hour to build, waiting on E2E tests gets much worse.

In my composite sketch, the problem was never that the E2E tests had bad coverage. The problem was relying on them delayed the release and forced developers to work overtime. Delayed

releases and slow bug fixes are neither good for the user nor good for the developer.

Even if the testing pyramid only gets you a B in terms of quality and coverage, while the E2E strategy gets you an A - I don't believe that's true, but will assume so to make a point - is going from a B to an A worth it if it takes you twice as long, for example?



Jferrin July 24, 2015 at 8:22:00 AM PDT

Doesn't this stance contradict your insistence that Google is user focused? How does this approach reflect that mantra? It seems you think that users only care about getting that new feature as fast as possible. This has been shown not to be true in numerous studies. There is a balance, I will grant that. Customers will not wait forever for a new feature, especially when dates were given ahead of time that need to be pushed out, but they will be a far happier customer, one more likely to expand their relationship if what you deliver does what they want and does it without being interrupted by a string of minor defects that can be resolved quickly when found. Your ability to fix quickly is meaningless in a business model where Land and Expand is integral to success.

Your position in this post paints a very negative position for e2e tests which I fear will be taken out of context by VPs everywhere and ruin product quality everywhere because gosh-darn-it Google says e2e testing is bad and doesn't help, so we aren't going to do it.

The decision of what to test and to what degree should be driven primarily from information. What sort of analysis do you do on escaped defects and how does that drive the test efforts and test types. I have witnessed on more than one occasion a defect that was not caught by existing unit and integration tests and made it to the field because a decision was made that time constraints dictated that the e2e tests could not be run in full. Those e2e tests would have caught the defects in question. Defects that cost the company far more in operations, Tech Support and ultimately dev time then it would have had they done the tests up front and that is on top of the ruined reputation with the customer base and the negative costs associated with that. Maybe Google doesn't care as much because of the nature of the relationship they have with their users. We after all do not buy your software. Your money comes

from ads mostly. I'll even concede that in your case you are probably right to have your outlook. Users of mobile apps, users of browsers, and internet-based apps EXPECT failure and so are more tolerant. My opinion is that people responsible for quality should be embarrassed by that. Instead, it looks like we embrace it and use it as an excuse to allow the continued release of shoddy code just to get it in the hands of your customers a few days early.

All that being said, I agree that Unit Tests and Integration tests must be done and are the foundation for all tests going forward. Having Developers responsible for quality and equal partners in delivering on quality and tests is essential to success, but the best unit and integration tests in the world will let bugs out the door that good e2e tests would catch. The important thing is to continually observe your results and the impacts to the customer/end product and target test improvements based on that knowledge. Maybe you agree with that, maybe you don't but your post makes it seem like we should turn our back on e2e and that is just not a good idea. Please read all hostility as passion, not aggression.

Reply



Jonty... April 28, 2015 at 2:35:00 AM PDT

Nice article Mike -

While I agree in principle of IT and Software delivery, am not sure if am board with this statement : "Although end-to-end tests do a better job of simulating real user scenarios, this advantage quickly becomes outweighed by all the disadvantages of the end-to-end feedback loop"

Have we reached a maturity level, where in software building process has become so much standarized and defects more predictable ?

I would argue there are whole lot of systems which still puts user feedback, by simulating end user flows high on the pedestal than faster feedback.

One key reason for E2E tests is because simulating all aspects of user behavior (the fundamental reason of the application) is too tedious at units level

It is great to see most org adopting matured and faster dev practices, but jumping into it without setting the house right for me is the biggest risk :)

I however subscribe to your thought, building layered Architecture is the need of the hour :)

[Reply](#)



Unknown April 29, 2015 at 8:03:00 AM PDT

Thanks for sharing your experience.

I'm new to TDD. I'm reading "Growing Object-Oriented Software, Guided by Tests" by Steve Freeman. The author has very interesting argument for end-to-end tests:

"Running end-to-end tests tells us about the external quality of our system, and writing them tells us something about how well we (the whole team) understand the domain, but end-to-end tests don't tell us how well we've written the code. Writing unit tests gives us a lot of feedback about the quality of our code, and running them tells us that we haven't broken any classes—but, again, unit tests don't give us enough confidence that the system as a whole works."

So I understand this statement as end-to-end tests give us feedback and tells whether we are moving in the right direction. After reading your post I got feeling that end-to-end tests are a waste of time. Don't you think they play vital role in the early stage of development?

[Reply](#)

▼ [Replies](#)

Anonymous May 1, 2015 at 12:57:00 PM PDT

You can have bad E2E tests that externally simulate things real users don't do; just because a test is E2E doesn't necessarily mean it represents the user. You can also have good unit tests driven by user scenarios, which test the specific task a unit would be given for a particular user scenario, as opposed to testing the unit as some abstract entity.

Quality and user impact are measured in both visible and invisible

ways. A bug where an implementation of equals() is broken could easily break the entire system and have a severe user impact. However, it's obviously harder to visibly explain the impact of that bug in terms of a specific user scenario or a specific E2E test.



Unknown March 12, 2018 at 2:44:00 AM PDT

Unit test in general will never give you feedback about the external quality. You can have a wonderfully written class with unit test which is wired wrongly into a system, and then you have nothing, but a false positive:

E2E is the only place where you can phrase external constraints.

Still it can be valid to decrease the amount of E2E because of specific reasons, but I unit testing cannot be alternative for E2E in any aspects.

[Reply](#)



Anand May 3, 2015 at 7:01:00 AM PDT

Got your point. I think for finance domain application stakeholders give more importance to E2E automated tests more as they want to ensure End User Experience or Customer Journeys meet the expected behavior. These tests not necessarily serve the purpose when designed badly and generally concentrate on proving something works. They are under a wrong pre-text that you can replace manual tests with these E2E tests.

[Reply](#)



Federico Rampazzo May 5, 2015 at 9:36:00 AM PDT

I feel like the title is misleading. I disagree with the title but I agree completely on the article

E2E tests are important - but you can't rely ONLY on them.

E2E tests are good for quality assurance, unit and integration tests are an aid to developers.

[Reply](#)

▼ [Replies](#)



Gene May 31, 2016 at 7:27:00 AM PDT

Completely agree. Title should be "E2E coverage VS velocity".

[Reply](#)



Gaurav Joshi May 6, 2015 at 3:29:00 AM PDT

What are your suggestions for legacy systems? Benefits of automated end to end tests are much larger than unit testing or acceptance testing. For new functional development, I completely agree with your approach.

At the moment, we are concentrating automating end to end regression manual tests to cut down our release cycle. We plan to add integration/unit testing to identified problem area. Could you suggest alternative approach.

[Reply](#)

▼ [Replies](#)

Anonymous May 6, 2015 at 9:18:00 AM PDT

Buy a copy of Working Effectively with Legacy Code by Michael Feathers :) Beyond that, you should measure progress not by whether you have a pyramid or not, but relative to where you were before. Even if you won't have a proper pyramid for a long time, but does it look more like a period today than yesterday?



Gaurav Joshi May 7, 2015 at 10:09:00 PM PDT

Will do. We were not targeting for pyramid but wish to achieve that during the journey. So for a legacy system which has no unit test coverage, what would be your suggestion?

1. Write E2E tests (we will use robot framework)- It would give us the most benefits
2. Write unit tests - Faster feedback for very low coverage.
3. Write test for subsystem which encompasses lot of classes and represent a fairly big unit of work - Use some thing like approval testing

Please ignore if the book already answers these questions. BTW I

did not understand "but does it look more like a period today than yesterday?" , Can you please elaborate?



Unknown May 2, 2016 at 9:08:00 AM PDT

On a legacy code project we were on with initially no unit tests, we made it our practice to write unit tests for all new classes that were added (preferably with TDD). When we needed to change an existing class, we would do that TDD style by adding tests for changes. We would do the minimal required to that class to pull out dependencies into the primary constructor being used, create a new constructor in which we could pass in mocks and stubs, and test against that. These refactorings were generally low risk as we only did the minimal changes required. We gradually added more and more coverage this way. And the classes that never changed were at a lower risk of breaking and were OK not initially being unit tested.

[Reply](#)



Adoniram Mishra May 6, 2015 at 8:41:00 AM PDT

I too am a believer of a test pyramid and just to add I believe in not repeating the test i.e if something can be tested at a lower level, push it to the lower level and try not to have the same validation at higher level. Also, We should aim for ~100% unit test code coverage as unit tests are first and most strongest line of defence.

[Reply](#)



Shadab Ansari May 7, 2015 at 8:41:00 AM PDT

Can I post this article in my blog by giving you due credit? It is really an eye opener for QA managers.

[Reply](#)

▼ [Replies](#)

Anonymous May 7, 2015 at 10:15:00 AM PDT

That's fine, as long as you both link to the original article and give due credit as you said.

[Reply](#)



Gaurav Joshi [May 7, 2015 at 10:03:00 PM PDT](#)

What are your thoughts on acceptance testing ? They are E2E in nature

[Reply](#)



Michael McDermott [May 11, 2015 at 7:23:00 AM PDT](#)

Do you think working in a dynamically typed language (such as Python or Ruby) changes the arguments here in some way?

[Reply](#)

▼ [Replies](#)

Anonymous [May 11, 2015 at 8:53:00 AM PDT](#)

The fundamental argument is the same. Additionally, you may need a few more unit tests to guard against things that normally would be caught at compile-time with a statically typed language.

[Reply](#)



itinsley [May 11, 2015 at 7:52:00 PM PDT](#)

I think a lot of posters are ignoring the importance of letting your tests drive your design. Thinking about how you are going to test your code encourages you to design good abstractions in your classes and services and should allow you to test business processes at the unit or integration level. When the tests exist with a close relation to the function or process the tests are likely to stay relevant and up-to-date. Having worked on a team that had extensive (many 1000's) of Cucumber E2E tests we ended up in a situation where engineers were maintaining tests while being unsure if the tests were still actually relevant or simply legacy remains. Because they are E2E by definition it is hard to define ownership of these tests in relation to any particular codebase, library or service and they end up as poorly maintained

'common' code with no individual feeling they have the right to delete them. Inevitably the tests continue to grow and build times get out of hand. If you are doing TDD using E2E tests the results can be disastrous with logic scattered around all over the code base.

By all means, have E2E tests but keep them broad and shallow - i.e. the 10% described in the article.

[Reply](#)



Unknown June 12, 2015 at 4:09:00 PM PDT

How about this? Stop blaming on the E2E Test Methodology, but blame on you, the developers for now doing a good job. I think developers are not capable if they break 10 things to fix 1 thing. Coming from defense background, I see that developers of the web technology don't seriously take responsibilities and accountability. If you are likely to break features that already worked from before, maybe it's time that go back to school.

[Reply](#)



OZ July 21, 2015 at 7:34:00 PM PDT

I can't believe it was written by Google engineer... It's like promoting of approach "my module works" - when you use unit-tests only, everything can work fine but not in collaboration. How it's not obvious for that Google engineer? I'm sure it even sounds offensive for a lot of Google engineers, especially for those who work on e2e-testing tools. See how many downloads one of them have in npm:

<https://www.npmjs.com/package/protractor>

It's the most awful article what can be found in this blog.

[Reply](#)

▼ [Replies](#)



Unknown April 22, 2017 at 4:47:00 PM PDT

Woah! hold your horses and cool your engines! He is not saying throw away E2E tests. He is only talking about the right balance. His initial analogy using the Big-O notation explains it very well. The fast turn-around time is very important. CD is very important. It not only helps to deliver new features but also faster bug fixes. The

above pyramid could give you a B grade quality, but an A grade turn around. However, E2E can not guarantee, an A++ quality. What does that translate to? Well imagine you are testing a passenger plane. If it is E2E tested, but when you are in 30,000 feet, and have a fault, you are going to crash before a fix is delivered to you. However, if you have a mechanism that identifies the problem quickly and the fix is delivered to you while you are airborne then you are in much better shape.

Thanks Mike for your fantastic article!

[Reply](#)



Ran Davidovitz August 2, 2015 at 1:12:00 PM PDT

Couldn't agree more, and i would even AIM higher than 70% UT code, all those E2E testing are killikng organization in over complicated and failing tests.

The E2E test should do a flow of UI to see items are connected and not broken

And no matter what, Keep the E2E code in the team's repo and not external repo!

[Reply](#)



Unknown September 9, 2015 at 9:07:00 AM PDT

What about refactoring? Isn't harder to continuously evolve an OOD when every class has a corresponding unit test? (every time you throw a class away you throw away the corresponding unit test and then write new tests for the new replacement class(es)).

With end to end tests, the core design of a software can be refactored (as often as it takes) without the need to refactor the tests (if the user facing API is the same).

Not to say that end to end tests are better than unit tests but I think that refactoring is a very frequent activity in agile software development and should be taken into account when comparing different testing approaches.

[Reply](#)



Mark Maxey October 17, 2015 at 12:47:00 PM PDT

I agree with the pyramid in theory, but not in not always in practice. When working on large legacy systems with no automated tests, I recommend inverting the pyramid. No one has budget or time to backfill unit tests. Transforming manual testing organizations means taking what they have and improving it incrementally. E2E automation and later integration shows fast ROI for management to fund unit automation for new and modified features.

[Reply](#)

▼ [Replies](#)



Unknown November 22, 2015 at 3:25:00 PM PST

Hi Mark I worked on projects with this characteristics: large codebase but lack of coverage. All attempts to bring quality and speed into the development that used E2E tests as a start seemed to fail due still poor quality of the code.

If the code isn't easy enough to write Unit tests, then I see some ideas to get some good results in weeks:

- Start writing integration tests for the most important components.
- In parallel start refactoring the code of these components / writing Unit tests.

If developers don't write Unit tests is because they don't know. If they ever know how to, I'm sure they will enjoy and be more productive since they can verify their code within seconds rather than hours / days.

At the end of the day ROI is about product quality and speed of development. Management should ignore the rest and focus on the product itself.

[Reply](#)



Mark Maxey November 27, 2015 at 5:03:00 PM PST

The systems I work on are multi-million line code bases that have existed for decades. These systems have static well-defined interfaces with other systems in our enterprise. This means that we can write E2E tests against the interfaces without being coupled with the implementation (good or bad).

With such large code bases, hundreds of developers who don't understand anything about API design or automated unit tests, and a fixed budget, schedule, and features, using the pyramid as recommended requires years of multi-discipline cultural change. I believe that change starts with writing E2E tests simulating externals and asserting results is a cheap way to verify the basic functionality of the systems from the perspective of the externals. One or two people is all you need to start the revolution and to show immediate ROI in terms management cares about (externally visible system function). Extending the revolution to unit tests as recommended is a huge investment for behavior below management's radar - a very hard sell.

[Reply](#)



Juan Mendes March 4, 2016 at 1:58:00 PM PST

My gist of the pyramid is: Do not try to cover edge case tests in end-to-end tests. For example, client side validation, grids without data, DB down, network out. Ideally you can test edge cases on the unit level. When you can't, you may end up with an extra end-to-end test. However, for every feature, there should be a few cases where you mimic the user in a typical usage scenario which makes sure the that unit tested parts to work when they come together.

[Reply](#)



Unknown March 9, 2016 at 11:20:00 AM PST

I'm not 100% convinced.

If a developer introduces a bug in the login or save functionality, I definitely want most of the end-to-end tests to fail. Something is very very wrong!

But. There definitely needs to be a detailed suite of unit tests in existence around logging in and saving! So the bug should also break at least one or two unit tests.

So: focus on the unit tests first.

If you have a *lot* of e-2-e test failing and no corresponding unit-test failing, the problem is probably that you are missing some unit tests. If possible write one or more unit test that captures the issue. Code coverage tools can help a bit. More often than not, after adding missing tests (which should initially fail, since they are meant to capture a bug that only surfaced in e-2-e) and then fixing the unit test failures, the majority or all of e-2-e tests will pass again. There are obviously e-2-e test failures that cannot be found in a unit test environment. When that's the case you definitely want the failing e-2-e test in your suite!

Also, the idea of shipping if say 90% of e-2-e tests pass, sounds ludicrous. If the failing tests are out-of-scope, take them out, or replace them with something that passes. Shipping with "10%" e-2-e test breakage means you don't have a good mental model of what you're shipping. So throw away the offending tests if you need to, but for every test you throw away, you should be able to determine whether it means that you are ditching some features, or need to prevent some edge cases, or that the tests were not (or no longer) valid.

Automated e-2-e tests are a great thing. You don't necessarily have to apply them to every build if that slows you down. They are definitely more brittle than unit tests. That's because there are a lot more moving parts in a e-2-e test than in a unit test! Same as real life :-)

Good e-2-e tests can protect against tricky regressions, where a lot of moving parts are involved.

Also, in your scenario of doom, you have a list of things that happen, and completely derail your planned release.

That the release gets derailed is a GOOD thing. I don't want to ship code if it was tested against moving targets / instable environments.

I definitely want to delay the release of a developer bungled the login functionality/

Those are all valid reasons for stopping the show.

The e-2-e tests that stop the show when things like that happen are your lifeline :-)

[Reply](#)



Unknown March 16, 2016 at 5:09:00 PM PDT

This comment has been removed by the author.

[Reply](#)



Unknown March 17, 2016 at 8:02:00 AM PDT

Hmm. This article seems to be implying that e2e tests will cause your development cycle to explode unless you abandon them in favor of faster unit and integration tests. I'm all down for smaller tests but I don't think having an e2e suite is going to kill you. It just shouldn't be the only line of defense you have.

In the fake scenario the devs lost over three days because they were apparently helpless to see if the changes they made to the code were good or not until after they got the results back from the e2e test suite. Most devs I know have some kind of Docker or Vagrant sandbox where they can see their change in action and can run at least some kind manual testing right at their desk. This doesn't catch everything but it would mean the three days "wasted" because they didn't know their fix was bad is a little out of bounds. I also think the day lost to hardware failure in the test lab is exaggerated too. That maybe happens once every few years unless you have the most crappy and complicated test setup in the world.

Other than flaky tests, it seems that all the issues in this article are less from having too many e2e tests and more from not having enough unit tests or a proper development environment. It's true that devs will still need to wait for their code to be deployed until after all the e2e tests have finished (and passed) but that doesn't mean developers can't get feedback from other sources before that and fix issues they find. Also, adding a little logging to your e2e tests makes it a billion time easier to track down why a test failed. Just sayin.

[Reply](#)



Unknown March 25, 2016 at 5:44:00 AM PDT

What is your recommendation when using an Agile approach? In Agile, testing is unit by unit. How do we test the whole flow in a large project? Using unit testing won't let us know if things will work when everything is completed.

[Reply](#)



Unknown April 1, 2016 at 2:39:00 AM PDT

Article assumes a lot of things about the way development is done and does have valid points on true agile development/testing organization, but this is not the case in many organizations around the world.

Google does great products and can be seen as one of trend definers in software development, but still world is not just around google or other similar hi-tech companies and I hope that no one takes the views in this article as a single truth of how development/testing is or should be done...instead, it provides very narrow and limited view!

I had a privilege as a consultant to witness the variety of different type of development organizations. Why things were done in a certain way was in many case due to the nature of the developed application or because of the history (15 years ago it was not so mandatory to create unit tests and a lot of products with this burden still exists) and in many cases the challenge was not in the feedback cycle and e2e tests were extremely valuable.

[Reply](#)

Anonymous April 14, 2016 at 12:37:00 PM PDT

I think that this article has many valid points but some invalid ones. It treats E2E as evil and a avoidable tasks. In my experience all tests are important in their timeframe in the development process: unit testing when writing the software, integration testing when the feature is ready and can be integrated with other components, and E2E testing. E2E testing is very useful to detect those intangible bugs, components alone can work perfectly (and thats what unit testing help to accomplish) but once they are delivered the workflow of an application can be incomplete, not user friendly, or simply be wrong.

[Reply](#)



AKASH DAS @feelings May 3, 2016 at 3:57:00 AM PDT

Hey Mike I am working for Target and I am busy nowadays convincing my leadership that we should bring API testing in place specially for products where the UI is evolving and the UI is not stable.

As we are centralized testing team and there are some other module specific testing teams also.

There are two questions from Leadership :

- 1) Is the bug detection count going to increase as result of API testing.
- 2) If the module teams are doing the API testing then when centralized testing team check the flow from startpoint to end point; how will that differentiate us from them in terms of testing differently and value addition.

As per you what are the answers for them .

Thanks in Advance.

[Reply](#)



Irtiza_Rizvi May 5, 2016 at 8:05:00 AM PDT

Never say NO to more e2e tests. Everyone agrees we need e2e tests because unit tests & integrations tests are not reliable. THEY MISS BUGS which seriously impact the user. Why put so much time in them when we can put equal time creating effective e2e tests which WILL CATCH bugs. This discussion will always continue when we allow developers to write/discuss about Testing. Developers only look for themselves when discussing how bad e2e tests are.

[Reply](#)



axel22 May 13, 2016 at 2:26:00 AM PDT

What went wrong:

- The team did not have a hermetic environment for their integration tests.
- The team did not run their integration tests before merging in their changesets.
- The team did not remember that they can actually revert a changeset

that broke the tests.

- The team failed to realize that debugging failed integration functionality takes even more time than debugging a test scenario (debugging sucks, testing rocks, remember? ;)).
- The team failed to write sufficiently many unit tests _in addition_ to end-to-end tests.
- The team was using flaky end-to-end tests.
- The team was using end-to-end tests that took too long.

[Reply](#)



axel22 May 13, 2016 at 2:36:00 AM PDT

I read the whole blog post now, and although the title sounds provocative, a colleague of mine pointed out the "more" keyword in the title. I agree that there should be a balance between unit tests and e2e tests, but solid e2e tests must still exist.

[Reply](#)



Unknown May 18, 2016 at 11:29:00 PM PDT

Unfortunately the real world isn't that easy. The possible number of tests increases when I combine units to modules and modules to applications. And most often applications have interfaces to other applications so the number of possible tests increases again. I agree that all types of tests in the pyramid are necessary. But it is not possible to give a rate like 70/20710 in general. Some people state I have 75% test coverage for example. If you ask them how they measure this coverage then then refer to executed lines of code. But in reality their test coverage is much smaller as the complexity increases with integration. So the art is to find the right unit tests, the right integration tests and the right e2e tests. You will always have to apply a risk based approach to find the right tests.

[Reply](#)



Gene May 31, 2016 at 7:41:00 AM PDT

Absolutely misleading and damaging title !

You should name it differently... "E2E coverage VS velocity" or "E2E trade offs" or something like that.

Article itself is a collection of materials from other blogs and articles ?

If you have problem with execution speed, there are tons of ways how speed them up:

- parallelize your tests
- manage them properly with suites (execute only tests that touches the area, which has been affected by the change).
- use "hybrid" test framework approach. For example: use API calls for the test preparation, instead of doing it via UI.

If you have "flaky" tests, then, 95% of the time, it is lack of tester's skills on how to design robust tests.

UI(E2E) tests are as useful as any other tests if done properly, and must be used along with unit and API tests in the right proportion and preferably in "hybrid" framework.

[Reply](#)



Sanjay June 28, 2016 at 6:00:00 AM PDT

Nice article to have understanding of testing pyramid. Regarding junit v/s integration test, I am really confused about having a worth of integration test. As with junit you are going to test only one unit at a time and second unit will fully mocked for all its behavior. Now when I mocked all the behavior of second unit for first unit, creating a integration test will not make difference communication between two objects are already test by mocking all scenarios. So in that case should we really opt for integration test ?

Thanks,

[Reply](#)



Unknown June 29, 2016 at 4:42:00 PM PDT

I think this article really deals with larger, enterprise projects. Smaller projects, particularly those with a great deal of success hinging on user interactions, benefit greatly from end-to-end interface driven testing. I can see how in a larger project they may lose value in many scenarios.

[Reply](#)



srehan July 26, 2016 at 8:05:00 AM PDT

Its really hard to release product without end to end tests when code base is complex. Unit and integration tests are great place to start but E2E tests combined all those different components and make test them. We find more tests in E2E than unit and integration tests, imagine a phone release without E2E testing how well it will work? There are lots of ways of speeding up testing cycle and better designed tests can run in minutes rather than hours or days. Perfect strategy is Unit and integration tests gating the master branch where nightly automated system tests kick in and find rest of issues..

[Reply](#)



srehan July 26, 2016 at 9:54:00 AM PDT

Doesn't fit for every product as the products which end up in real user and have are complex need to be tested at E2E. There is lots of overhead in maintaining integration tests, on the other hand with good automation framework E2E tests can be very simple to add but test coverage can be great. E2E testing taking long time is not excuse not to do it as it can be speed up to matter of hours instead of days or even minutes in some cases. A right balance between pyramid testing is very much needed, I guess pyramid structure is ideal for small projects but not for very complex software.

[Reply](#)



Unknown November 14, 2016 at 12:18:00 PM PST

One thing end to end tests can do is to help your manual testers identify areas that need their eyes. It's very true E2E can be flakey and can be slow. Rather than having those tests hold up CI or cause a fire to fight in dev land, use them as a supplemental testing tool. Data for your testers to test better or areas to hit hard before release.

Remove the focus on the machine finding bugs and instead use it as a tool for the only users you have internally, your manual testers.

That said I do completely agree with the pyramid approach. Just some extra food for thought on how to deal with e2e test results.

[Reply](#)



Unknown February 19, 2017 at 7:57:00 AM PST

Is the testing pyramid a good strategy for testing?

Every time I read about or discuss this thing, it seems to fuel more confusion, not less.

For example, why do the labels here differ from the original? Are unit, integration and e2e all the same classification?

Shouldn't we be thinking about testing in a pipeline instead?

[Reply](#)



Louis CAD February 26, 2017 at 10:18:00 AM PST

Meanwhile, there's no performance tests for Google Docs, as scrolling performance is horribly slow (I use it on a MacBook Air 2013, 8GB of RAM, Intel Core i5). Same for the Google Play Games app, but worse (about 2fps when scrolling as images are processed on the UI/rendering thread)

[Reply](#)



Blogger84 May 23, 2017 at 3:31:00 AM PDT

I agree with most of the arguments but there is another point of view. If we treat E2E as pure functional tests, they give invaluable quality confidence before pushing stuff to QA environment. QA team can have their own set of cases but since you have already worn the hat of the QA guy, you are less likely to face bugs which can not be caught in 'Partial' Integration Tests that you mentioned or in unit tests. Note that I am still for extensive unit test coverage but not so much for integration tests. So basically, hour glass shape is not as bad in some cases.

[Reply](#)



gleb bahmutov May 30, 2017 at 7:31:00 AM PDT

Things really change if the economics of tests change. What if E2E tests were as quick and simple to run as unit tests? Then the entire pyramid would flip! I call such changed pyramid "testing trapezoid"
<https://glebbahmutov.com/blog/testing-trapezoid/>

At Cypress.io (which I have joined recently) we are working hard to make web browser tests fast, reliable and repeatable. For us, it makes sense to write more E2E during development, because ultimately they reflect user's behavior better.

[Reply](#)

▼ [Replies](#)

[gabriel](#) June 26, 2017 at 6:45:00 AM PDT

You may have an incredibly fast and reliable web driver, it won't make web pages load faster in the browser.

[Reply](#)



[Unknown](#) July 14, 2017 at 2:57:00 AM PDT

I know this blog is a few years old, so I'm wondering if you changed your stance on this at all?

From reading the post, it looks like there are (or were) other big problems that the post doesn't explicitly recognise:

- introducing broken code the day before release date
- blocked testing effort due to the bug being called a "failed test"
- there appears to be an acceptance of "flaky tests" being ok?
- the "automation triangle" has been mislabelled as a "testing triangle", but doesn't represent the full picture of testing (i.e. doesn't include investigative/exploratory testing at all). In fact, the whole post only talks about automation, which can only assert an explicit expectation. What about the rest of the testing activities that focus on exploration and investigation?
- The only types of risks being recognised here appear to be "integration" risks. No other types of risks that should be tested for are mentioned here.

I wonder if these problems have been picked up and resolved within google since this blog was written?

[Reply](#)

▼ Replies

Roy Adams October 15, 2017 at 1:22:00 PM PDT

A more recent TotT from Sept 21, 2016 reiterates many of the same points but takes a softer stance than "just say no to e2e tests":

<https://testing.googleblog.com/2016/09/testing-on-toilet-what-makes-good-end.html>

Reply



Francois August 3, 2017 at 6:00:00 AM PDT

The whole end to end testing description fail to explain the full picture and downgrade the need for end-to-end testing?

I disagree here. What is failed to describe is how all different ind of testing fit like a lego blocks into one another and not just a pyramid of start with unit tests and work oneself up to lesser set of E2E testing.

One people need is a traceability matrix, showing first foremostly the functional and non-functional requirements. For each of those linked this to all the unit tests - in a matrix per system/sub-system/sub-component.

Then for each of these there are integration testing, service level testing.

Both unit test and service testing can be automated. Most cases these are semi-automated depends upon the complexity of the content and user data available.

Now end to end testing make sure the integration tests are working. This is the first step in true integration testing. Not merely a service test but test that the integration of all the applications and components integrate correctly.

Only here after the full end to end testing commence.

So you really do four more than just three layers.

1. Unit Tests

2. Service Tests
 3. Integration Tests
 4. End to End Functional Testing
 5. Regression Tests
 6. Performance Tests
 7. Automated Tests
- and so on.....
5. End to End Non-Functional Testing

So I totally disagree with the article that done away with end to end testing or making it less. Also the percentages proportionally are incorrect. Merely because one can for each unit test have a like for like functional end to end test and/or non-functional end to end test.

So E2E testing is definitely not a risk it on the contrary minimize risk for implementation not met the requirements.

[Reply](#)



Unknown August 31, 2017 at 3:51:00 PM PDT

How do you make sure you are not doing a replication of effort when integration and unit testing are done?

[Reply](#)



Dave Schinkel October 6, 2017 at 1:35:00 PM PDT

You totally failed to mention TDD and design feedback.

[Reply](#)

▼ [Replies](#)

Anonymous May 2, 2019 at 1:26:00 AM PDT

Agreed. End to end tests or system functional tests or whatever you want to call them are of value when you develop features test first (in my opinion).

Reply



Revathi May 10, 2018 at 3:56:00 AM PDT

Hi,

I am tasked to create an automated tool for android system events. Can you advise me which automated testing tool can I use to create testing/generating system events? Can Robotium, appium or Espresso be used? In my understanding robotium and appium is useful for UI testing but can we use that for system event testing?

[Reply](#)



Unknown September 24, 2018 at 9:55:00 PM PDT

Hi Mike,

Here is my interpretation of Test Pyramid covering all aspects of testing from risk-perspective.

<https://amtoya.com/blogs/test-pyramid-as-a-risk-filter/>

Regards,
Amit

[Reply](#)



Dherik March 8, 2019 at 5:58:00 AM PST

I worked on an application that have more than 12 hours of end-to-end tests (that we later managed to distribute the test on different machines and reduce the time, but this is another story). I can only agree with the author.

Even being a monolith application (what it was easier to put up and running to test) it was nightmare to maintain the tests. Most part of the time we was maintaining the tests instead of catching bugs. Discover the origin of a bug on a end-to-end test takes a lot of time. We also dealt with a lot of "false-negative" tests and few time to understand the problem and correct it: Java Applet loading problems, expected element not found on the page (plus other problems about the speed automation), maintain query code that are

just used on the database memory test (because the original use database specific code), etc.

[Reply](#)



Kenowi June 28, 2019 at 3:35:00 AM PDT

In an ideal world I would agree with the pyramid of testing as proposed by Google long time ago, but most companies do not see themselves as 'software' companies like google. They should, but they don't. That brings you to the question, if you would have limited time/budget would you prefer unit tests or e2e-tests? For the first one you need developers and finally you do not know if your application works, for the second you can have non-developers maintain them and you actually know that your main features work. So it's all about taking a risk on how long things will need to be maintained. E2Etests is product insurance, Unit tests is maintenance insurance. Short vs long-term vision.

[Reply](#)



Unknown July 15, 2019 at 3:37:00 AM PDT

Ok, so how exactly creating unit tests benefits a project in case of regression? If a SOLID principles are met, then unit tests won't show regression. On the other hand, integration and E2E tests would. I see TDD as a tool to design a software, not to test it. They force a developer to apply good practices, but if a piece of software is complete and follows good practices, the test will never fail, because if we need to change some feature, we would remove this piece (with tests) and write it from scratch to meet new requirements (and of course provide unit tests for this new piece).

TDD is the option, not the requirement to create a piece of good software, unit tests written after creating a code are useless, so without TDD there's no need for unit tests (of course I'm still assuming that the software is designed well).

So if we don't do TDD, we won't meet this funny "pyramid" and we shouldn't write ANY tests? That's some serious bullshit...

[Reply](#)



smackyou November 21, 2019 at 10:27:00 AM PST

how would unit tests catch a front end ui workflow error?

[Reply](#)

▼ [Replies](#)



Unknown February 26, 2020 at 4:54:00 AM PST

There are many frameworks which can mock the service calls and create you the exact payload (can be a simple json file of payload) and can test all your UI screens and controls. Even UI has unit tests and integration tests frameworks available.

[Reply](#)



Unknown February 26, 2020 at 4:56:00 AM PST

This is a great read. It is difficult to convince your QM on the same as they feel real user like test (end to end) is only the best one to define the quality of software but actually as per the pyramid shown more tests in the lower one makes the quality of software better.

[Reply](#)



M December 17, 2020 at 7:39:00 AM PST

If your E2E Tests aren't fast and reliable. You aren't doing it right. It's not rocket science. but if you need a better strategy and approach reach out. I'll give you some ideas.

[Reply](#)

[Add comment](#)

New comments are not allowed.



